

5-消息存储与检索服务实现

核心能力

- **消息持久化**: 单聊/房间消息同步写入 MySQL `message` 表, 幂等键 (`session_id`, `client_msg_id`)。
- **历史拉取接口**: HTTP `/api/message/history` 按 `session_id` + `anchor_seq` 倒序分页返回。
- **前端适配**: `web_demo/static/single_chat.html` 进入会话自动拉取历史, 缓存去重、滚动到底部。

后端改动

- **表结构**: `sql/schema.sql`
 - `message(msg_id PK unique, session_id, msg_seq, sender_id, msg_type, content_json, timestamp_ms, client_msg_id)`
 - 索引: `PRIMARY(session_id, msg_seq)`, `UNIQUE(session_id, client_msg_id)`, `UNIQUE(msg_id)`, `idx_session_time(session_id, timestamp_ms)`
- **持久化 DAO**: `logic/message_dao.{h,cpp}`
 - `Insert(const PersistMessage&, std::string* err)`
 - `List(session_id, anchor_seq, limit, std::vector<PersistMessage>*, std::string* err)`
- **HTTP**: `logic/http_server.cpp`
 - `/api/message/send` / `/api/room/message/send`: 分配 seq (Redis), 写 MySQL, 再更新 Redis 未读/快照并推 Kafka。
 - `/api/message/history`: 校验会话归属 (单聊需包含当前用户, 房间需在成员内), 分页拉取 MySQL 历史。
- **gRPC**: `logic/grpc_service.cpp` 上行消息分支同样落库。
- **注入**: `logic/main.cpp` 创建 `MessageDao` 注入 HTTP/GRPC。

Redis/seq 说明 (保持原有键位)

- 单聊 seq: `msgid:<uid_small>:<uid_large>` 通过 `INCR`。
- 房间 seq: `msgid_group:<room_id>` 通过 `INCR`。
- 仍写入: `session:last_seq:{sid}`、`user_sessions:{uid}`、`unread_count:{uid}:{sid}`、`user_session_meta` 等。

HTTP 历史拉取接口

- 路径: `POST /api/message/history`
- 鉴权: Header `Authorization: Bearer <token>` 或 `Token: <token>`
- 请求示例:

```
{
  "session_id": "s_1:2",
  "anchor_seq": 0,
  "limit": 50
}
```

- 说明: `anchor_seq` 为游标, 取 `< anchor_seq` 的记录; 0 表示最新开始。limit 默认 20, 最大 200。
- 响应示例 (按时间正序返回, 便于直接渲染) :

```
{
  "code": 0,
  "message": "ok",
  "data": {
    "messages": [
      {
        "msg_id": "msgid:1:2-1",
        "session_id": "s_1:2",
        "msg_seq": 1,
        "sender_id": 1,
        "timestamp_ms": 1730000000000,
        "msg_type": "text",
        "client_msg_id": "c123",
        "content_json": "{\"msg_id\":\"...\", \"msg_seq\":1,...}"
      }
    ]
  }
}
```

前端变更 (web_demo)

- 文件: `web_demo/static/single_chat.html`
 - 进入会话时调用 `/api/message/history` 拉取历史; `historyLoadedSessions` 避免重复首拉。
 - 消息缓存去重 (同 seq 或同 client_msg_id) 。
 - `currentMaxSeq` 跟踪并用于已读推进; 滚动到底部自动 `mark_read`。

使用建议

1. 运行前执行 `sql/schema.sql` 初始化/更新表结构。
2. 启动 logic 服务后, 前端单聊页自动拉取历史; 若需分页上滑可继续传 `anchor_seq=当前最小seq`。
3. Kafka 仍用于实时推送; 持久化默认同步直写 MySQL, 可后续旁路/切换为 Kafka 消费写库。

后续可扩展

- 补充 WebSocket 拉历史 (复用 HTTP 接口) 或在接入层转调。
- 为房间/单聊历史做上滑分页加载。
- 增加内容裁剪/大小限制或拆表分片。